

Luna Park — Interactive 3D Amusement Park

Project Report — Interactive Graphics Course

Enrico Battistoni

Davide De Blasio

2026-06-20

Table of contents

1	Project Introduction	2
1.1	Overview	2
1.2	Motivation and Design Goals	2
1.3	Technology Environment	2
1.4	High-Level Feature List	2
2	External Libraries and Assets	3
2.1	Libraries	3
2.2	External 3D Models	4
2.3	Textures and HDR Maps	4
3	Technical Implementation — Rides and Animations	4
3.1	Scene Graph and Hierarchical Models	4
3.2	Ferris Wheel — Counter-Rotation	5
3.3	Carousel — Phase-Offset Horse Bobbing	5
3.4	Roller Coaster — Parallel Transport Frame	5
3.5	Tagada — Compound Oscillation	6
3.6	Other Rides	6
4	Technical Implementation — Lighting, Shaders, and PBR	6
4.1	Day/Night Cycle	6
4.2	Artificial Night Lighting	6
4.3	Water Shader (Custom GLSL)	6
4.4	Sky Shader (Custom GLSL)	7
4.5	Fireworks Shader (Custom GLSL, GPU Particles)	7
4.6	PBR Materials	7
5	NPCs, Camera, and Interactions	7
5.1	NPC Visitors	7
5.2	Camera System	7
5.3	Controls Reference	7
6	Conclusion	8
7	References	8

1 Project Introduction

1.1 Overview

Luna Park is an interactive 3D amusement park rendered in real-time via Three.js and WebGL 2.0. The user freely explores a virtual park containing seven rides (Ferris Wheel, Carousel, Roller Coaster, Tagada, Panoramic Train, Hot Air Balloons, Shooting Gallery), controls them via in-world panels, adjusts the time of day, and can enter first-person view aboard any ride. The project demonstrates the core topics of the Interactive Graphics course within a single cohesive application.

1.2 Motivation and Design Goals

The amusement park theme was chosen because it naturally motivates every major graphics technique in the syllabus. Each ride is a hierarchical model whose compound rotations produce characteristic motion. All animations are hand-coded in JavaScript (no imported clips). The park contains several custom GLSL shaders (water, sky, fireworks) and a structured lighting system with a full day/night cycle.

1.3 Technology Environment

Three.js r170 provides the scene graph abstraction and WebGL renderer. All custom shaders are written in GLSL ES 3.0 (`#version 300 es`). The render loop uses `EffectComposer` with `RenderPass` → `UnrealBloomPass` → `OutputPass`. Tone mapping is `ACESFilmicToneMapping`; shadows use `PCFSoftShadowMap`.

1.4 High-Level Feature List

Category	Features
Rides	Ferris Wheel (counter-rotating gondolas), Carousel (phase-offset horse bobbing), Roller Coaster (spline track, energy-governed speed), Tagada (compound oscillation), Panoramic Train (CatmullRom ring track), Hot Air Balloons (wind-drift with 2D noise), Shooting Gallery (pointer-lock FPV aim)
Environment	PBR-textured ground, asphalt paths, fence with string lights, food stalls, stage, river with animated water shader, trees and vegetation
Lighting	Day/night cycle (sun/moon orbit), HemisphereLight, PCF shadows, 12 auto-lampposts, per-ride decoration lights with colour picker
Sky	4 HDR equirectangular presets (night/sunrise/day/sunset) crossfaded by a custom two-texture sky shader
Post-processing	Selective UnrealBloom, ACES filmic tone mapping
NPCs	10 procedurally animated visitors with A* pathfinding and 14 outfit variants
Particles	GPU fireworks with 3 burst types (spherical, corona, willow)

Category	Features
Camera	Click-to-fly, 6 preset viewpoints (keys 1–6), FPV aboard any ride, OrbitControls
HUD	Time-of-day arc, digital clock, manual slider, auto-advance toggle, ride hotbar



(a) Ferris Wheel



(b) Carousel



(c) Roller Coaster



(d) Tagada

Figure 1. The four main rides of Luna Park.

2 External Libraries and Assets

2.1 Libraries

Library	Version	Role
Three.js	r170 (MIT)	Rendering engine: scene graph, geometries, materials, lights, WebGL renderer
OrbitControls	r170 (MIT)	Mouse orbit, pan, zoom
EffectComposer + RenderPass + UnrealBloomPass + OutputPass	r170 (MIT)	Post-processing pipeline
GLTFLoader	r170 (MIT)	Load GLB 3D models
RGBELoader	r170 (MIT)	Load HDR environment maps
tween.js	23.1.3 (MIT)	Smooth eased transitions (camera fly-to, ride ramp, lamppost toggle)

No build step is required: the browser resolves all `import` statements via native ES modules served by any HTTP server.

2.2 External 3D Models

All models are in GLB format (binary GLTF 2.0). A custom `loadGLB()` wrapper immediately discards embedded animation data to enforce the policy that every motion is hand-written JavaScript.

Model	Source
Ferris Wheel, Carousel Horse, Hot Air Balloon Roller Coaster Track	Quaternius free pack (CC0) “Animated roller coaster” by assetfactory, Sketchfab (CC0)
Wacky Worm (Train)	Poly Pizza (CC0)
Carnival Shooting Booth	Sketchfab (CC0)
Human characters (14 variants)	Quaternius “Ultimate Animated Character Pack” (CC0)
Environment props (trees, bushes, rocks, benches)	Quaternius environment pack (CC0)
9 mm Pistol and Bullet	Sketchfab (CC0)

2.3 Textures and HDR Maps

PBR texture sets (albedo, normal, roughness, AO) for grass, asphalt, and wood are from ambientCG (CC0). Four RGBE equirectangular HDR maps (night, sunrise, day, sunset) are from Poly Haven (CC0). All assets are CC0 or MIT; the project is fully distributable.

3 Technical Implementation — Rides and Animations

3.1 Scene Graph and Hierarchical Models

The scene is structured as a tree of `THREE.Object3D` nodes whose transforms compose from parent to child. Every ride encapsulates its entire sub-tree. For example, the Tagada hierarchy:

```
tagada (Group)
  base
  armPivot (Group)      pitch + roll + yaw
```

```

    arm
      armExtension (Group) telescoping for boarding
      disc (Group)    continuous spin
      platform
      seat[0..7]    with passengers
    controlPanel

```

This allows compound oscillation by writing to three different rotation properties on three nodes, with children inheriting all parent transforms automatically.

3.2 Ferris Wheel — Counter-Rotation

Each gondola is a child of a mount Group that co-rotates with the wheel ring. The gondola mesh is rotated by exactly the opposite angle about the same axis:

$$\text{worldRot}(\text{gondola}) = \text{parentRot}(\text{mount}) + \text{localRot}(\text{gondola}) = (+\text{wheelSpin}) + (-\text{wheelSpin}) = 0$$

The gondola's world orientation is identically zero — always upright, exactly as a real Ferris wheel behaves.

3.3 Carousel — Phase-Offset Horse Bobbing

Eight horses are mounted at equal angular spacing around a rotating platform. Each horse i bobs vertically with:

$$y_i(t) = \text{BASE_Y} + \text{BOB_AMP} \times \sin(2\pi \times \text{FREQ} \times t + i \times 2\pi/8)$$

The uniform phase distribution produces a travelling Mexican-wave pattern around the ring.

3.4 Roller Coaster — Parallel Transport Frame

The rail centre-line is extracted from the GLB mesh by computing the centroid of each 24-vertex ring, producing 395 raw points that are filtered and down-sampled to 80 control points. A closed `THREE.CatmullRomCurve3` with `arcLengthDivisions = 20000` interpolates the track.

A **Parallel Transport (Bishop) frame** is used instead of the Frenet frame (which flips at inflection points). The frame is initialised with the rail spoke direction closest to world-up, then propagated along the curve by applying the rotation that maps each tangent to the next. Two trains of four carriages each run simultaneously, offset by half a circuit.

Cart speed varies with altitude:

$$v^2 = v_0^2 + 2g(y_{\text{top}} - y)$$

The ride cycles through four states: STOP → LAUNCH → COAST → BRAKE → STOP, with a 3-second station pause.

3.5 Tagada — Compound Oscillation

Simultaneous three-axis oscillation of the `armPivot` Group:

$$\begin{aligned}\text{pitch} &= \text{offset} + A_p \sin(\omega_p t) \\ \text{roll} &= A_r \sin(\omega_r t) \\ \text{yaw} &= \text{continuous spin}\end{aligned}$$

The pitch (2.6 rad/s) and roll (1.9 rad/s) frequencies are incommensurable, producing quasi-periodic Lissajous motion. The disc spins independently at 2.0 rad/s.

3.6 Other Rides

Panoramic Train: Follows a hand-authored CatmullRom curve (20 control points) circling the park perimeter at scenic height, with curvature-based banking. **Hot Air Balloons:** Three balloons drift via 2D value noise, producing smooth organic trajectories, with vertical sine oscillation and configurable wind. **Shooting Gallery:** Uses the Pointer Lock API for FPV aim; raycasts against targets trigger score updates with distance-based multipliers and a 30-second timer.

4 Technical Implementation — Lighting, Shaders, and PBR

4.1 Day/Night Cycle

A normalised time parameter $t \in [0, 1]$ governs the diurnal cycle (0.00 = midnight, 0.25 = sunrise, 0.50 = noon, 0.75 = sunset). The sun position is:

$$\text{sunAngle} = (t - 0.25) \times 2\pi, \quad \text{sunY} = \sin(\text{sunAngle}), \quad \text{sunX} = -\cos(\text{sunAngle})$$

The `HemisphereLight` sky colour transitions from clear blue (0x87ceeb) to deep navy (0x2a3a64). The sun colour animates from orange-gold at dawn to near-white at noon. A moonlight `DirectionalLight` (0x4466aa, intensity 0.40) activates opposite the sun. Tone-mapping exposure and background colour are animated as functions of `sunHeight`.

4.2 Artificial Night Lighting

Four categories of artificial lights: 12 lamppost `PointLights`, a stage `SpotLight`, 6 path `SpotLights`, and per-ride decoration `PointLights` with phase-staggered sinusoidal pulsing and runtime colour configurability via the HUD picker.

4.3 Water Shader (Custom GLSL)

A fully custom `ShaderMaterial` with GLSL ES 3.00 stages:

- **Vertex:** Sum of four Gerstner wave trains with different directions, wavelengths, and amplitudes. Surface normal computed analytically from partial derivatives.
- **Fragment:** Depth-based colour (deep blue → shallow cyan), procedural caustics via two-scale value noise, Blinn-Phong specular, Schlick Fresnel blended against sky tint, crest foam, and moonlit tint at night.

4.4 Sky Shader (Custom GLSL)

An inverted-sphere `ShaderMaterial` that samples two HDR equirectangular textures simultaneously. The vertex stage removes the view matrix translation and forces `gl_Position.z = gl_Position.w` (far plane). Four presets are crossfaded on a 24-hour schedule via the `uMix` uniform.

4.5 Fireworks Shader (Custom GLSL, GPU Particles)

Ballistic integration in the vertex shader:

$$\text{pos} = \text{center} + \text{velocity} \times \text{age} - \frac{1}{2} \times \text{gravity} \times \text{age}^2$$

Particles exceeding lifetime are culled by projection to a degenerate position. Three burst archetypes: **spherical** (uniform sphere), **corona** (equatorial ring), **willow** (drooping trails).

4.6 PBR Materials

All `MeshStandardMaterial` instances use the GGX/Trowbridge-Reitz microfacet BRDF:

$$f(l, v) = \frac{c_{\text{diff}}}{\pi} + \frac{F(l, h) G(l, v, h) D(h)}{4 (n \cdot l) (n \cdot v)}$$

Texture channels: albedo (sRGB), normal (tangent-space), roughness, metalness, AO. All use maximum anisotropic filtering (16×).

5 NPCs, Camera, and Interactions

5.1 NPC Visitors

Ten visitors roam the park using a **WAIT** ↔ **WALK** state machine. Pathfinding uses A* on a coarse occupancy grid (CELL = 2 world units) with string-pulling to smooth routes. Each NPC has a four-layer procedural gait: foot targets with Hermite swing arcs, two-bone analytic leg IK, pelvis rhythm (bob, lateral shift, yaw counter-rotation), and trunk/arm swing. Fourteen outfit variants plus HSL hue shift per NPC provide visual variety.

5.2 Camera System

Four modes: **Orbit** (default, `OrbitControls` with damping), **Click-to-fly** (raycaster → 1.2 s TWEEN), **Preset viewpoints** (6 hardcoded positions via keys 1–6), **FPV** (camera attached to ride nodes, ESC exits with 0.8 s tween).

5.3 Controls Reference

Input	Action
Left-click on ground/object	Fly camera to point (1.2 s)
Left-click on ride's control panel	Start/stop ride
Left-click on lamppost	Toggle lamppost (Auto/Off/On)
Keys 1–6	Preset viewpoints
Key T (near shooting gallery)	Enter FPV aim mode
Key F	Launch fireworks show
ESC	Exit FPV / aim mode

Input	Action
Space	Pause/resume day/night
HUD ride speed sliders	Adjust ride speed ($0.2\times$ – $1.5\times$)
HUD time slider	Set time of day
HUD colour picker	Change decoration bulb colour
Bottom hotbar buttons	Enter FPV for a specific ride

6 Conclusion

Developing Luna Park was a substantial effort that integrated nearly every topic from the Interactive Graphics course into a single working application. The most challenging aspects were the roller coaster's parallel transport frame (ensuring correct cart banking without Frenet-frame flipping) and the procedural NPC gait engine (eliminating foot-skate across 14 skeleton variants). We are satisfied with the result: the park is visually rich, runs at 50–60 FPS on consumer hardware, and offers a wide range of interactive experiences. The complete source code and a live demo are available at <https://sapienzainteractivegraphicscourse.github.io/final-project-tokteam>.

7 References

1. Three.js r170 documentation. <https://threejs.org/docs/>
2. tween.js (23.1.3). <https://github.com/tweenjs/tween.js>
3. Quaternius CC0 3D model packs. <https://quaternius.com>
4. Poly Haven CC0 HDR environment maps. <https://polyhaven.com>
5. ambientCG CC0 PBR textures. <https://ambientcg.com>